

Đại học Quốc gia Thành phố Hồ Chí Minh

Đại học Khoa học tự nhiên

Khoa Công nghệ thông tin



HOMEWORK

Automation Testing

Subject **SOFTWARE TESTING**

Class **22KTPM3**

Student **22127374 – Lê Thanh Tâm**

Ho Chi Minh City, 2025

Table of Contents

1	Group Task Allocation	2
2	Introduction	2
2.1	Selected Features for Testing	2
2.2	Data Generation Tool	2
3	Automation Strategy and Implementation	2
3.1	Test Architecture Design	3
3.1.1	Modular Structure (Test Suites, Setup Test Cases)	3
3.1.2	Reusable Test Cases for Common Functions (e.g., Login)	3
3.2	Data-Driven Testing Approach	3
3.2.1	Test Data Design in Excel	3
3.2.2	Data Binding in Katalon Studio	3
4	Checkpoint and Assertion Strategy	3
4.1	Usage of Verification Keywords	4
4.2	Dynamic Verification Logic based on Test Data	4
5	Feature 1 Automation: Admin - Products	4
5.1	Feature Overview	4
5.2	Automated Scenarios	4
5.2.1	Search Product	4
5.2.2	Add New Product	5
5.2.3	Edit Existing Product	5
5.2.4	Delete Product	5
5.3	Execution Results and Analysis	5
6	Feature 2 Automation: Admin - Orders	6
6.1	Feature Overview	6
6.2	Automated Scenarios	6
6.2.1	Search Order	6
6.2.2	Update Order Status	6
6.2.3	Update Status Message	7
6.3	Execution Results and Analysis	7
7	Bug Reporting	7
7.1	Summary of Discovered Bugs	7
7.2	Detailed Bug Reports	7
8	Conclusion & Demonstration	8
8.1	Summary and Conclusion	8
8.2	YouTube Video Demonstration Link	9

1 Group Task Allocation

Full name	Student ID	Feature
Luong Hoàng Dung	22127076	1. Checkout _Address 2. Contact Form Submission
Lê Thanh Tâm	22127374	1. Admin - Add product 2. Orders
Nguyễn Lâm Nhã Uyên	22127445	1. Admin - Login 2. Sign up
Lê Nguyễn Yến Vy	22127466	1. Change Password 2. Add product to cart

2 Introduction

2.1 Selected Features for Testing

Two core administrative features were selected for automation: Admin - Products and Admin - Orders. These features were prioritized based on their high business impact on the e-commerce application's functionality.

- **Admin Products:** This feature is critical for inventory management and involves complex forms with numerous data validation rules, making it an ideal candidate for a comprehensive, data-driven testing approach.
- **Admin Orders:** This feature is the heart of the sales workflow. It includes complex business logic, such as order status progression and data synchronization, which are vital to test for ensuring operational integrity.

2.2 Data Generation Tool

The automation scripts were developed and executed using a consistent technical environment to ensure reliability and repeatability.

- **Application Under Test (AUT):** The Toolshop
- **Target Version:** sprint5-with-bugs (Deployed locally)
- **Automation Tool:** Katalon Studio
- **Primary Language:** Groovy
- **Test Browsers:** Chrome, Firefox, Microsoft Edge

3 Automation Strategy and Implementation

This chapter details the technical approach and strategies employed to create a robust, maintainable, and data-driven automation suite for the selected features.

3.1 Test Architecture Design

The test architecture was designed around self-contained test cases, with each test case handling an end-to-end scenario for a specific user action.

3.1.1 Modular Structure (Test Suites, Setup Test Cases)

A monolithic approach was adopted where each test case (e.g., TC_AdminProducts_AddProduct) is a complete, self-contained script. Each script is responsible for the entire test lifecycle: opening the browser, navigating to the application, performing the login, executing the core test steps for the specific feature, verifying the outcome, and closing the browser. This structure ensures that each test case can be run independently without external dependencies.

3.1.2 Reusable Test Cases for Common Functions (e.g., Login)

For each of the seven Test Cases, a corresponding Test Suite was created (e.g., TS_AddProduct_Validation). The primary role of each Test Suite is to manage the data-driven execution for its associated Test Case. It connects the test case to its specific data source and executes the test case iteratively for each row of data. In this model, browser management (@SetUp/@TearDown) is not used at the suite level; it is handled within each test case iteration instead.

3.2 Data-Driven Testing Approach

A data-driven methodology was adopted to test numerous scenarios with varied inputs efficiently. This was achieved by separating the test logic (the script) from the test data.

3.2.1 Test Data Design in Excel

Microsoft Excel was used as the external data source. A single workbook (22127374_Data.xlsx) was utilized to centralize all test data. Within this workbook, the data was organized into seven separate sheets. Each sheet was named after and corresponded directly to one of the seven automated Test Cases (e.g., a sheet named AddProduct_Data was used for TC_AdminProducts_AddProduct). Each row in a given sheet represents a unique test iteration, containing columns for input data as well as control columns like **ExpectedResult** and **ExpectedMessage** to manage the dynamic verification logic.

3.2.2 Data Binding in Katalon Studio

The link between the Excel file and the script was established using Katalon Studio's built-in features. A **Test Data** object was created to connect to the Excel file. Corresponding **Test Case Variables** were declared in the test script to act as placeholders. Finally, the **Data Binding** feature within each dedicated **Test Suite** was configured to map the columns from the appropriate Excel sheet to the **Test Case Variables**.

4 Checkpoint and Assertion Strategy

Checkpoints are used after key actions to validate the application's state and determine the pass/fail status of a test case.

4.1 Usage of Verification Keywords

The primary mechanism for creating checkpoints was Katalon's built-in Verify keywords. A variety of these keywords were used depending on the validation requirement, including:

- WebUI.verifyElementPresent to confirm the existence of key elements after navigation and login.
- WebUI.verifyTextPresent to check for the presence of success or error messages on the page.
- WebUI.verifyElementNotPresent to confirm an item was successfully deleted.

4.2 Dynamic Verification Logic based on Test Data

A key feature of the assertion strategy was its dynamic nature. The test scripts contain an **if/else** control structure that reads the value from the **ExpectedResult** column in the data file. Based on this value (e.g., 'SUCCESS', 'FAIL_VALIDATION'), the script executes a different verification path. For a 'SUCCESS' case, it checks for a success message; for a 'FAIL_VALIDATION' case, it checks if the actual error message matches the one specified in the **ExpectedMessage** column. This allows a single, intelligent script to test multiple, varied outcomes.

5 Feature 1 Automation: Admin - Products

5.1 Feature Overview

This chapter provides a detailed overview of the automation process for the "Admin - Products" feature. It covers the scope of the automated scenarios developed and presents an analysis of the execution results. This includes the ability to perform full CRUD (Create, Read, Update, Delete) operations: adding new products to the catalog, viewing and searching for existing products, modifying product details, and removing products that are no longer available. Automating this feature is critical to ensure the integrity of the product database and the reliability of the e-commerce platform.

5.2 Automated Scenarios

To ensure comprehensive test coverage of the Admin - Products feature, four key scenarios were automated using Katalon Studio. These scripts were designed to be data-driven and include robust checkpoints to validate application behavior accurately.

5.2.1 Search Product

- **Scenario Description:** This test case automates the product search functionality. The script inputs a search term (e.g., product name) from a data file, executes the search, and verifies that the application displays the correct results. It also tests for cases where a search should yield no results.
- **Data-Driven Approach:** The script reads search terms, expected results (e.g., a specific product name to appear), and expected outcomes (e.g., SHOWS_RESULTS or SHOWS_NO_RESULTS) from an external Excel file.

- **Checkpoints/Verifications:** The checkpoint uses the **WebUI.verifyTextPresent** keyword. This function scans the entire web page to confirm that the expected text (either the name of a product for a successful search or the "No products found" message for an unsuccessful one) is present on the page after the search is performed.

5.2.2 Add New Product

- **Scenario Description:** This test case validates the product modification functionality. The script selects a product, navigates to the "Edit Product" form, enters new information from a data source, and saves the changes.
- **Data-Driven Approach:** The new details for editing and the expected success message are provided via an Excel file.
- **Checkpoints/Verifications:** The checkpoint relies on the **WebUI.verifyTextPresent** keyword. After submitting the update, the script scans the page to verify that the "Product saved!" confirmation message is present, indicating that the system has processed the action.

5.2.3 Edit Existing Product

- **Scenario Description:** This script automates the deletion of a product. It finds a specific product, clicks the delete button, and handles the confirmation alert.
- **Data-Driven Approach:** The name of the product to be deleted and the expected success message are provided from a data file.
- **Checkpoints/Verifications:** To validate the deletion, the checkpoint uses **WebUI.verifyTextPresent**. This keyword confirms that the "Product deleted" success message is displayed somewhere on the page after the product has been removed.

5.2.4 Delete Product

- **Scenario Description:** This script automates the deletion of a product. It finds a specific product, clicks the delete button, and handles the confirmation alert.
- **Data-Driven Approach:** The name of the product to be deleted and the expected success message are provided from a data file.
- **Checkpoints/Verifications:** To validate the deletion, the checkpoint uses **WebUI.verifyTextPresent**. This keyword confirms that the "Product deleted" success message is displayed somewhere on the page after the product has been removed.

5.3 Execution Results and Analysis

The execution of these four automated scenarios confirms that the basic CRUD functionalities of the "Admin - Products" feature are working. The data-driven tests successfully submitted various inputs, and the checkpoints verified the presence of expected confirmation messages on the page after each action. The successful automation of this feature provides a baseline regression suite for future development sprints.

6 Feature 2 Automation: Admin - Orders

6.1 Feature Overview

The "Admin - Orders" feature is a critical administrative function that allows staff to manage the entire lifecycle of customer orders. This includes viewing a list of all orders, searching for specific orders, and most importantly, updating an order's fulfillment status and associated messages. Automating these workflows is essential for ensuring that order processing is reliable and that status changes are correctly handled by the system, which directly impacts customer satisfaction and business operations.

6.2 Automated Scenarios

Three primary scenarios were automated for the Admin - Orders feature. These scripts were enhanced with comprehensive, multi-step checkpoints to ensure high confidence in the test results.

6.2.1 Search Order

- **Scenario Description:** This test case validates the search functionality within the orders page. The script is designed to input a search query from a data file, execute the search, and verify the outcome.
- **Data-Driven Approach:** Search terms and expected verification parameters are managed in an external Excel file.
- **Checkpoints/Verifications:** A robust checkpoint was implemented. Instead of a generic text search, the script retrieves a list of all result rows (<tr> elements) from the orders table. It then programmatically iterates through this list, checking the text of each row to confirm that a row containing the specific **VerificationParameter** (e.g., customer name or order ID) exists. This ensures the search result is present and correctly located within the results table.

6.2.2 Update Order Status

- **Scenario Description:** This script automates the process of changing an order's fulfillment status. It identifies an order, opens the edit view, selects a new status, and saves the change.
- **Data-Driven Approach:** The new status to be selected and the expected confirmation message are supplied by a data file.
- **Checkpoints/Verifications:** A comprehensive, two-step checkpoint was implemented:
 1. **Feedback Verification:** The script first waits for the success toast message to appear and verifies its text against the expected message, confirming the system's immediate response.
 2. **Data Integrity Verification:** The script then navigates back to the main orders list, dynamically locates the exact order that was just modified, and verifies that the text within its "Status" column has been updated to the new value. This confirms the change was successfully saved and is reflected correctly.

6.2.3 Update Status Message

- **Scenario Description:** This script tests updating both the order status and an accompanying custom text message. The script first locates a specific type of order (e.g., "ON_HOLD") before updating its details.
- **Data-Driven Approach:** The new status, the custom status message text, and the expected confirmation text are all controlled via an external data file.
- **Checkpoints/Verifications:** This scenario also uses a two-step checkpoint for maximum accuracy:
 1. **Feedback Verification:** It first verifies the success toast message to confirm the system processed the action.
 2. **Data Integrity Verification:** It then verifies that the custom text entered (`StatusMessageInput`) is correctly displayed in its designated area on the order detail page. This ensures the message data was persisted and rendered accurately. Additionally, the script is designed to fail explicitly if a prerequisite order cannot be found.

6.3 Execution Results and Analysis

The execution of the automated tests for the "Admin - Orders" feature demonstrates a high degree of reliability in the order management workflow. The use of comprehensive, multi-step checkpoints provides strong confidence in the results, as the tests validate both the immediate user-facing feedback (toast messages) and the final state of the data in the system. This approach ensures that backend processes, data persistence, and front-end rendering are all working in concert. The resulting regression suite is a valuable asset for maintaining the quality and stability of this critical business feature.

7 Bug Reporting

This chapter summarizes the critical defects that were discovered during the manual test execution phase and subsequently confirmed by the automated test suite. Full details for each defect have been documented in the accompanying 22127374_BugReport.xlsx file.

7.1 Summary of Discovered Bugs

7.2 Detailed Bug Reports

The automated test suites were executed against the "Admin - Orders" feature, revealing several critical defects. The failures logged by Katalon Studio provide repeatable, concrete evidence of significant issues. The following is a breakdown of bugs confirmed by each automated test case.

Bugs Confirmed by TC_AdminOrders_Search

The test suite `TS_SearchOrders_Validation` showed that the search functionality is fundamentally broken.

- **Defect: Search fails to return valid results.** The test case iteration for TC01 (searching for an existing invoice) failed. The script was unable to verify the presence of the invoice in the search results, indicating the search function does not correctly retrieve existing data.
- **Defect: No feedback for empty search results.** The test case iteration for TC02 (searching for a non-existent invoice) also failed. The script expected to find a "No results" message, but the text was not present on the page, leaving the user with a blank list and no explanation.

Bugs Confirmed by TC_AdminOrders_UpdateStatus

The test suite `TS_UpdateStatus_Validation` confirmed a critical flaw in the business logic of the order status workflow.

- **Defect: Flawed status transition logic.** When attempting to test an invalid status transition (reverting to a previous status), the script failed. The root cause was an `UnsupportedOperationException` because the script attempted to select a dropdown option that was disabled. This indicates the application's business logic for enforcing the sequential status workflow is not handled correctly, leading to script failure instead of a user-friendly error message.

Bugs Confirmed by TC_AdminOrders_StatusMessage

The test suite `TS_StatusMessage_Validation` revealed multiple defects related to input validation and error handling.

- **Defect: Improper API error handling on frontend.** When testing with a message at the maximum character limit, the backend returned a `422 Unprocessable Content` error. However, the frontend failed to handle this API response gracefully, instead displaying the raw, technical error object directly in the success message area: `[object Object] 422....`
- **Defect: XSS vulnerability in status message.** When testing with a string containing `<script>` tags, the system was expected to reject the input and show an "Invalid characters" error message. Instead, the application accepted the input and incorrectly displayed a "Status updated!" success message, confirming a security vulnerability.

8 Conclusion & Demonstration

8.1 Summary and Conclusion

This project successfully achieved its objectives of designing, developing, and executing a data-driven automated test suite for critical features of the Toolshop application using Katalon Studio. A robust architecture was implemented, separating concerns into browser management, reusable setup scripts, and data-driven test cases for specific functionalities.

The automation suite proved to be highly effective. It successfully validated the correct behavior of the application for positive test scenarios. More importantly, it consistently and reliably reproduced a number of high-priority defects. The failures recorded by the automated tests serve as concrete evidence of significant issues in search functionality, business logic, and data validation.

In conclusion, the developed test suite provides significant value as a regression testing tool, capable of ensuring the quality of the Toolshop application.

8.2 YouTube Video Demonstration Link

A video demonstration of the entire automation process, including script execution for the selected features, has been recorded and uploaded to YouTube. **Video Link:** [\[https://www.youtube.com/watch?v=uZ8BQq\]](https://www.youtube.com/watch?v=uZ8BQq)

No.	Criteria	Grade	Self-Assessed Grade
1	Name of Feature 1 (missing any of the following: "report", "script" or "video" results in 0 points)	50	50
	1.1 Report + Bug	15	
	1.2 Test cases	5	
	1.3 Script files	10	
	1.4 Data files	10	
	1.5 Videos	10	
2	Name of Feature 2 (missing any of the following: "report", "script" or "video" results in 0 points)	30	30
	2.1 Report + Bug	15	
	2.2 Test cases	5	
	2.3 Script files	10	
	2.4 Data files	10	
	2.5 Videos	10	
	Total	100	100